

Lab 12: Document and Bundle the Capstone

Maps to: A4, A6, A8

Produce a clean, testable release bundle with API docs, frontend assets and operational instructions.

AI Vibe Coding Assets

- ai_vibe.py - OpenAI Python SDK runner
- mock-data.json - synthetic request and test context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - first-run and refinement prompts
- working-files/generated/ - isolated AI output for review

First-Run Prompt

Review the complete FastAPI/SQLite/browser project and generate release documentation only. Include clean setup, test, run and verification commands; OpenAPI and environment notes; a bundle inventory; and a checklist excluding .env, databases, caches and virtual environments. Do not invent passing evidence.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
pytest -q
```

Detailed procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 7: Run the full test suite and save the terminal evidence.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 8: Review /docs and add missing summaries, tags, response descriptions and examples.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 9: Verify .gitignore excludes .env, *.db, caches and virtual environments.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 10: Create .env.example without live secrets.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 11: Follow README setup from a clean virtual environment.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 12: Zip only the documented source, tests and static assets.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 13: Run the acceptance checklist and record the final version identifier.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 14: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 15: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Acceptance criteria

A clean copy installs, tests, starts, serves the storefront/CRM dashboard, completes catalog/customer/order flows and contains no secret or database file.